



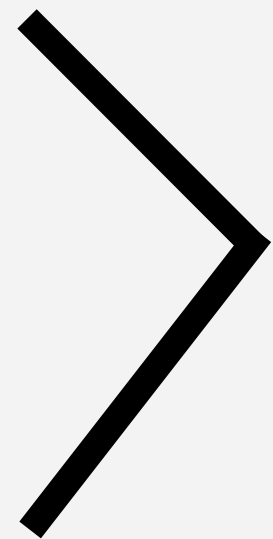
AI 與軟體工程整體趨勢

— AI 如何改變開發流程與工程實踐（全局整理） —

——
報告學生：蔡明憲 資訊碩一

參考論文：

- A. Generative Artificial Intelligence for Software Engineering—A Research Agenda (2025)
- B. AI-Driven Innovations in Software Engineering A Review of Current Practices and Future Directions (2025)
- C. Navigating the Complexity of Generative AI Adoption in Software Engineering (2024)





重點架構

影響範圍

AI 介入軟體工程哪
些活動？

現況證據

實務上主要用在哪、
好在哪、卡在哪？

效益與風險

效率提升之外，品
質/資安/合規的影響
是什麼？

落地關鍵

AI 要變成團隊能力，
流程上需要補哪些
機制？

AI 介入軟體工程生命週期各階段



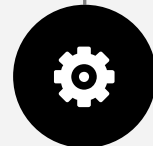
圖片來源: <https://arthvhanesa.hashnode.dev/the-6-stages-of-the-software-development-life-cycle-sdlc-explained>



需求: 整理訪談/回饋、生成 user story/SRS 草稿



設計: UML/架構草圖生成、權衡分析 (效能/擴展/安全)



實作: 程式碼生成/補全、API 推薦、註解/摘要

測試: 測試案例生成、覆蓋補強、缺陷/高風險模組預測

部署: 部署腳本/設定檔輔助、部署前檢查清單/風險提示

維護: 異常偵測 (預警)、修補/重構建議、文件同步更新



AI 在 SDLC 的實務現況

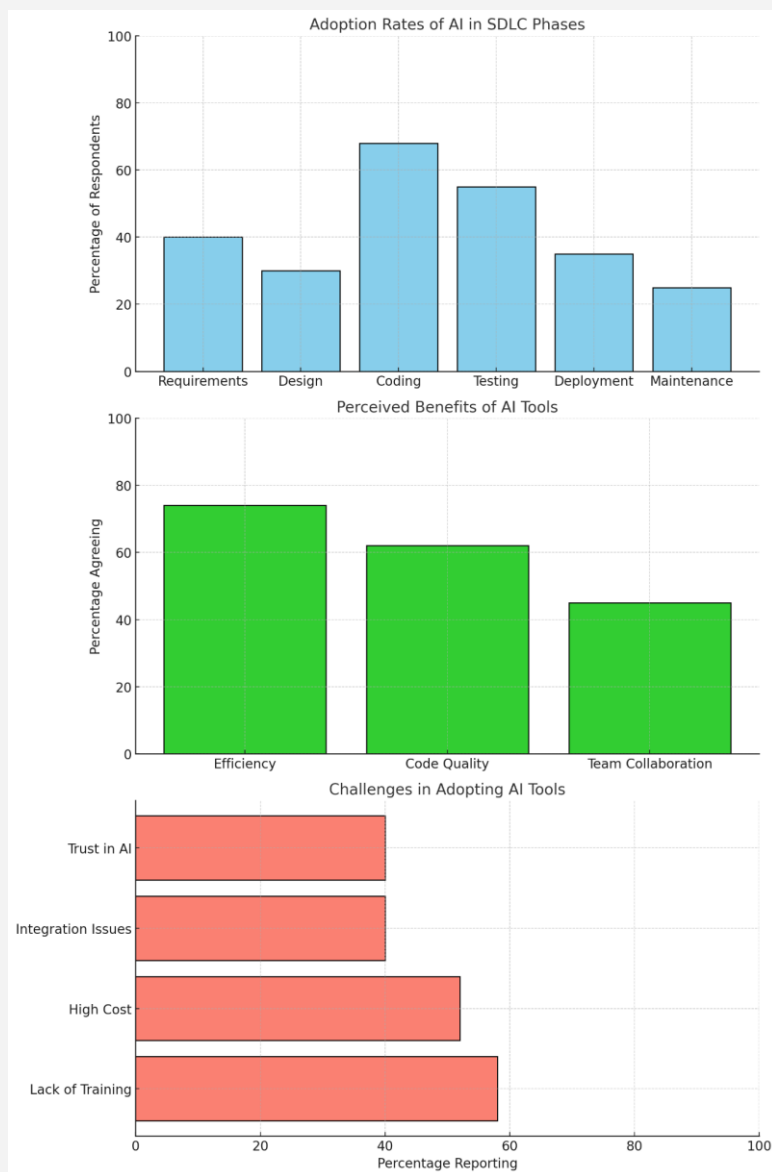


Figure 1. Analysis. (Paper B)

用在哪

Coding 最高，其次 Testing；
需求/設計/維護較低

好在哪

效率最明顯，其次程式碼品質；
團隊協作相對有限

卡在哪

缺乏培訓最常見，其次成本與整合問題；
信任仍是門檻

➤ **整體結論：** 工具能提升個人效率，但要擴散到全流程仍需訓練與整合機制。

生成式 AI × 軟體工程：研究議題全景

- 研究整理出 **78 個開放研究問題**，歸納成 **11 個議題領域 (tracks)**
- 這 11 個領域涵蓋**生命週期各階段**，也包含流程、管理、能力、教育、法律倫理等面向
- 含意：AI 對軟體工程的影響是「跨階段、跨角色、跨治理」的長期變化

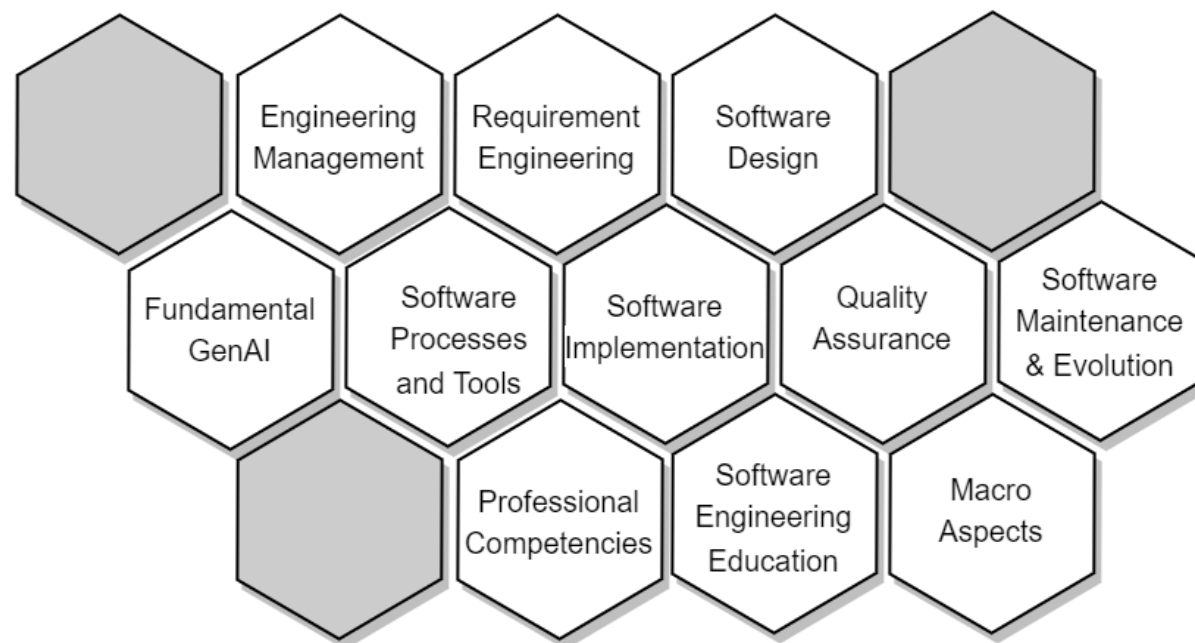


Figure 4: Research Agenda on GenAI for Software Engineering (Paper A)



軟體工程實踐的演變

軟體工程一路在追求 **更快交付** 與 **更多自動化**

- **流程導向 → 智慧導向：**
從管理流程 → AI 參與決策與產出
- **自動化升級：**
從 CI/CD 的測試/部署自動化 → 延伸到程式生成與維護
- **角色轉變：**
工程師從「編碼者」→ 更重視審查、整合與把關
- **維護進化：**
從被動修補 → 更預測、更多自適應

Table 1. Evolution of Software Engineering Practices. (Paper B)

Era	Methodologies	Key Features	Introduction/Adoption Period
Traditional	Waterfall model	Linear, sequential approach to software development	1970s
Iterative	Agile, Scrum	Flexible, iterative approach emphasizing collaboration	1990s
Modern	DevOps, continuous integration	Automated testing, continuous deployment practices	2010s
AI-Driven	AI-powered tools, machine learning	Automated coding, adaptive software maintenance	2020s

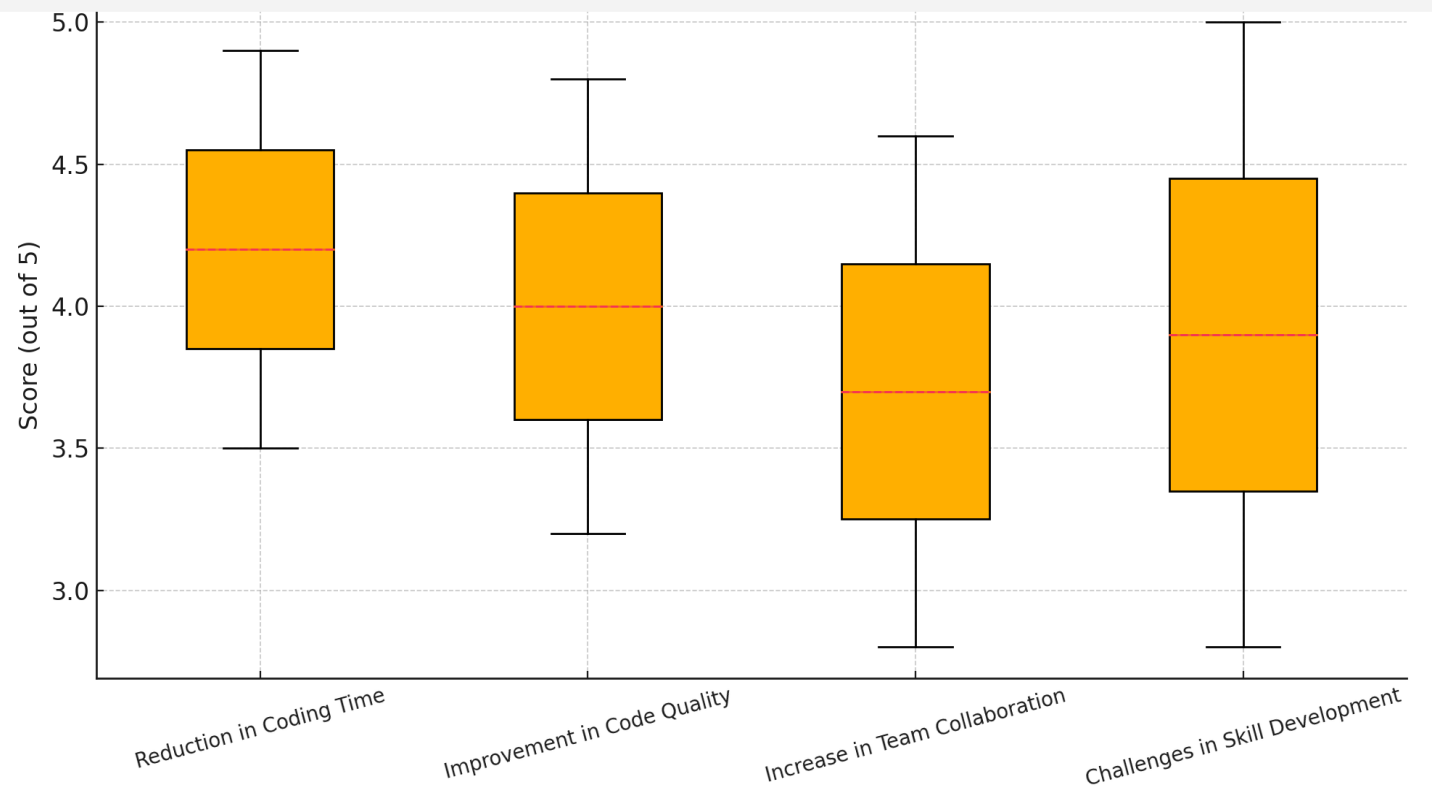


Figure 2. Key metrics evaluated in this study. (Paper B)

AI 工具影響的四個指標

- **減少編碼時間：**
共識最高 → 個人效率提升最明顯
- **程式碼品質提升：**
多數認同有幫助，但略低於速度
- **團隊協作提升：**
相對有限 → 仍偏個人生產力工具
- **技能發展挑戰：**
負向指標偏高 → 訓練與落地仍是瓶頸

AI 整合到軟體工程：三大挑戰

技術挑戰

- 資料品質/可用性不足（雜訊、不完整、隱私限制）
- 與既有工具鏈/舊系統整合困難（相容性、流程干擾）

倫理 & 法律

- 偏見風險（模型承接資料偏差）
- 智財/授權風險（訓練資料、生成程式碼的權利歸屬）

勞動力影響

- 技能落差（AI 素養、評估與驗證能力不足）
- 抗拒改變（信任不足、工作焦慮、習慣既有流程）

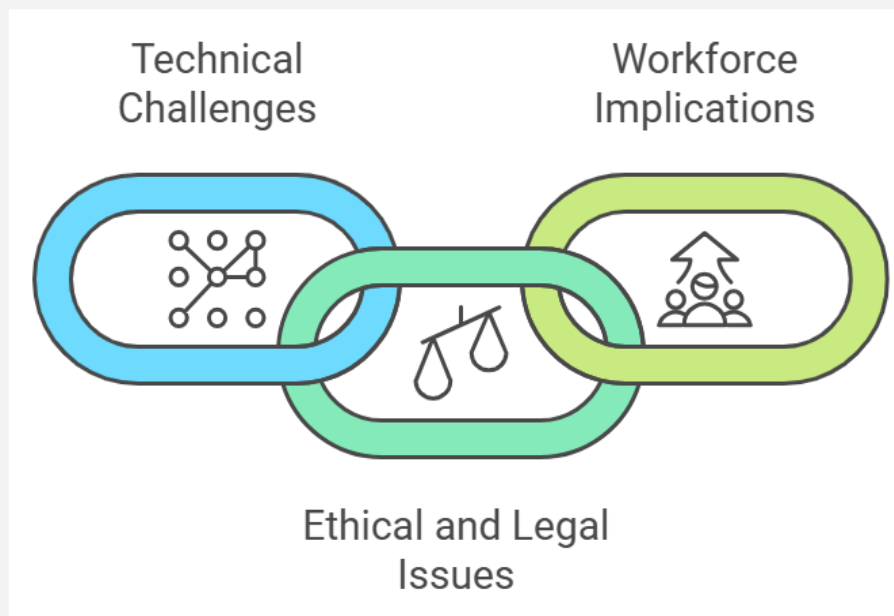


Figure 3. Challenges and considerations. (Paper B)

➤ **結論：**不能只「導入工具」，還要同時處理合規、訓練與把關。



AI 導入要補上的三件事

技術面 → 讓輸出可驗證

- 放進「審查流程」與「自動測試」
- 補齊上下文：規格、範例、規範、介面契約

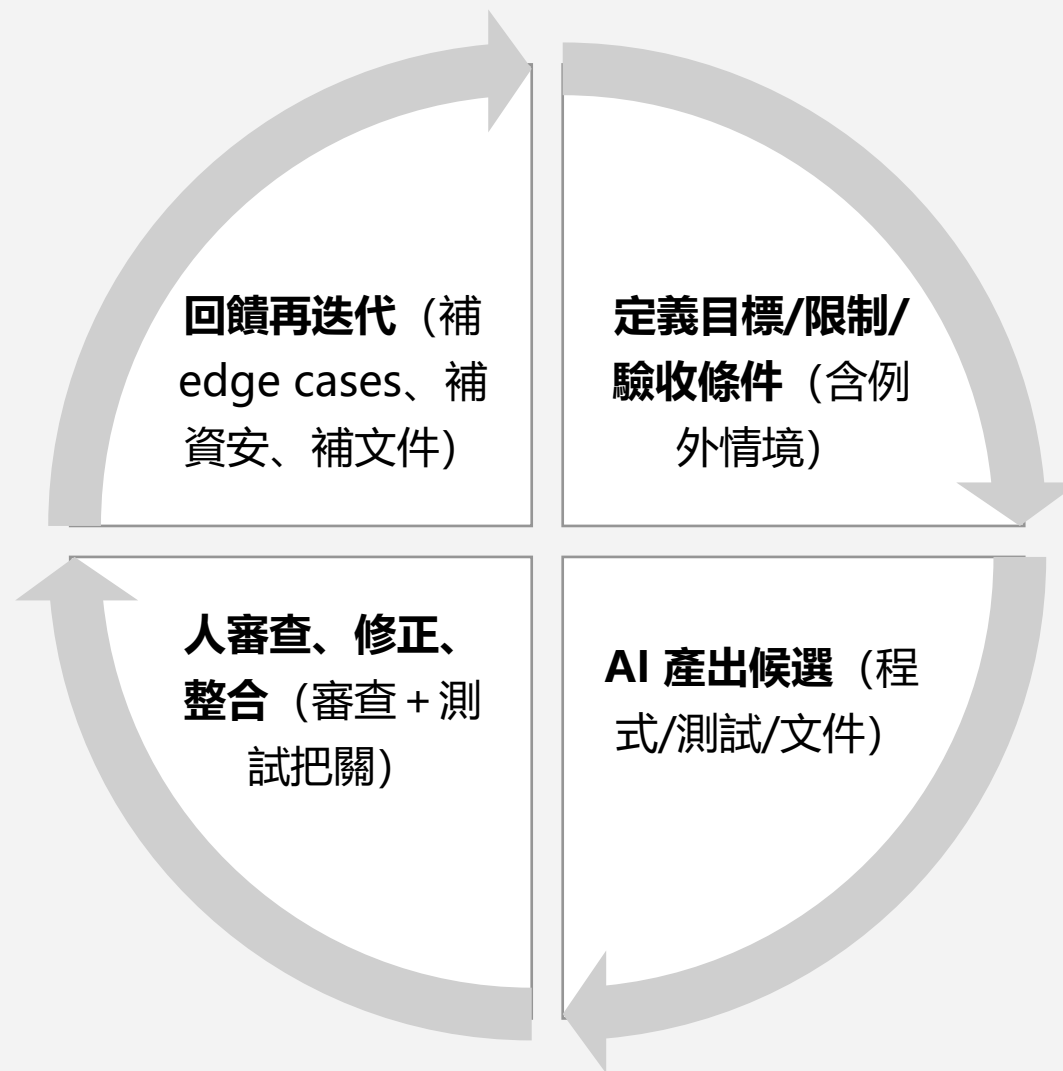
倫理法務面 → 讓使用可合規

- 資料使用界線清楚（敏感資訊、外部服務、留存規則）
- 生成內容可追溯（來源/版本/決策紀錄）

勞動力影響 → 讓團隊用得起來

- 訓練與共識：能用/不能用、怎麼審查
- 人類參與流程（Human-in-the-loop）：AI 產出 ≠ 直接上線

把 AI 放進流程後：工作會變成「人機迭代回路」





落地關鍵：AI 要變成團隊能力，需要接進既有工作流

審查與測試門檻要完整

- AI 產出也要走審查流程、測試門檻、回歸驗證。
- 重點是「能快速產出」但「不能跳過把關」。

工具鏈整合要做起來

- 版本庫、議題、建置/部署、監控要串起來。
- AI 才能真正融入日常流程。

資安與資料政策要先定義

- 哪些資料可用、哪些不可外流要講清楚。
- 包含外部服務使用規範、敏感資訊處理原則。

可追溯性要留得下來

- 至少能回溯使用情境、版本與決策紀錄。
- 出問題時才好定位與釐清責任。

HACAF 採用框架：GenAI 為什麼「能用」但「不一定會用」？

HACAF = Human-AI Collaboration and Adaptation Framework (人機協作與適應框架)

哪些因素會影響工程師的 **使用意圖 (IU / Intention to Use)**

- **PT 技術感知**：覺得有用、好用、相對優勢
- **CF 相容性**：能不能融入現有流程與工作習慣
- **SF 社會因素**：同儕影響、自我效能（會不會用、敢不敢用）
- **PEF 個人與環境**：個人創新性、組織支持（培訓/規範/資源）

➤ **結論**：採用不只看工具強不強，更關鍵是「流程相容性 + 組織支持 + 同儕認同」能不能到位。

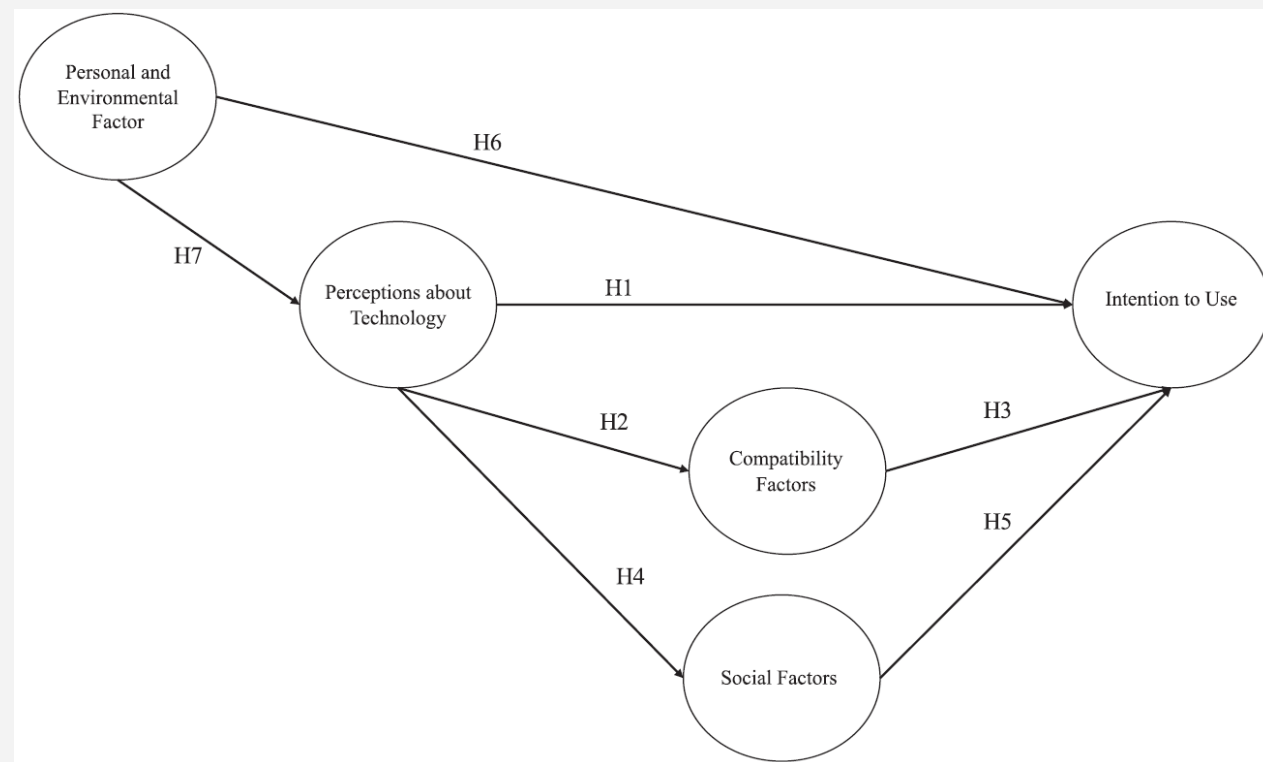


Fig. 1. The HACAF. (Paper C)

HACAF 實證結果 (1 / 2)

- **Coefficient (路徑係數)**：影響方向與強度 (越接近 1 影響越強)
- **p-value (顯著性)**：這條關係是否成立 (常用門檻 $p < 0.05$)
- **T Statistics (T 值)**：顯著性的強度 (常用門檻 $T > 1.96$)
- **R² (決定係數)**：模型解釋力 (0 ~ 1, 越高代表解釋越多)
- **St. Dev. (標準差)**：bootstrapping 下係數的波動 (越小越穩)

Table 13. Path Coefficients, Bootstrap Estimates, Standard Deviation, T Statistics, and p-Values (Paper C)

Hypotheses	Coefficient	Bootstrap Mean	St. Dev.	T	p
H1: PT → IU	0.155	0.150	0.122	1.271	0.204
H2: PT → CF	0.766	0.766	0.047	16.405	0.000
H3: CF → IU	0.536	0.537	0.090	5.936	0.000
H4: PT → SF	0.405	0.408	0.076	5.346	0.000
H5: SF → IU	0.087	0.090	0.064	1.373	0.170
H6: PEF → IU	-0.004	-0.004	0.056	0.064	0.949
H7: PEF → PT	0.313	0.317	0.072	4.313	0.000

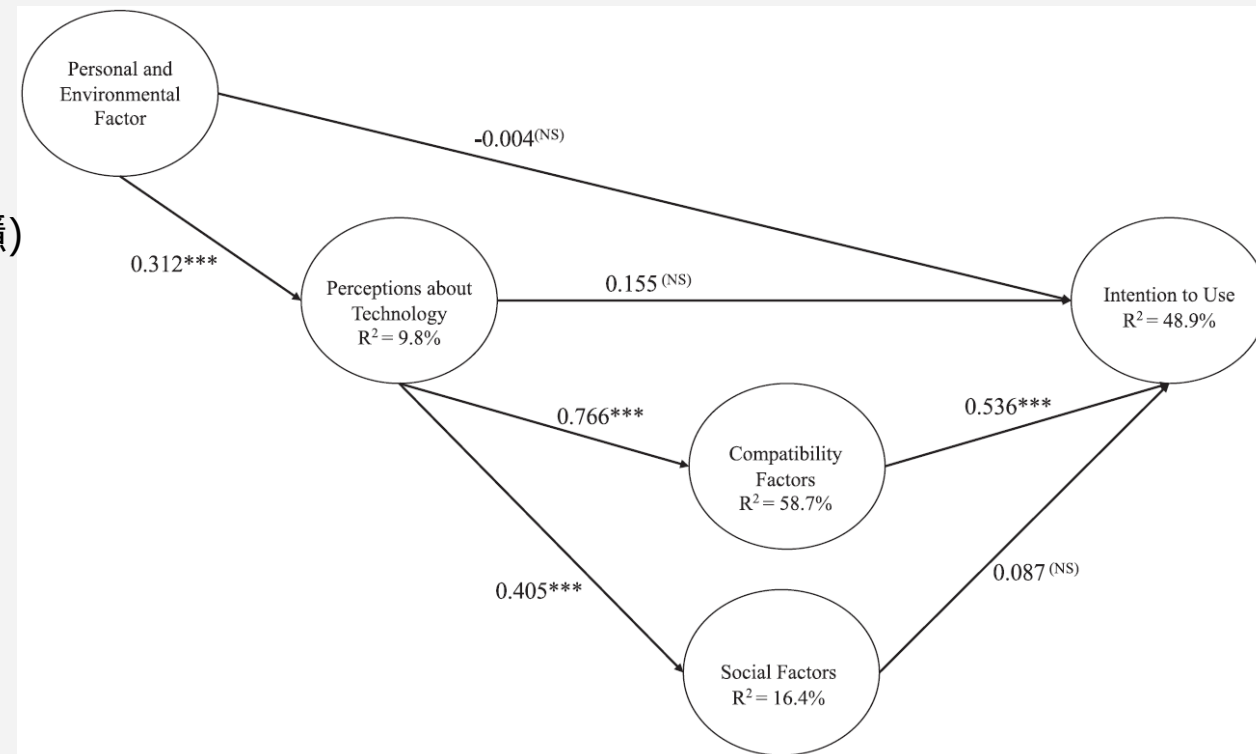


Fig. 2. HACAF' s structural model with R2 and path coefficients (*** $p < 0.001$, (NS) $p > 0.05$). (Paper C)

HACAF 實證結果 (2 / 2)

1. 相容性 (CF) 是使用意圖 (IU) 的最強驅動 (H3 顯著)
2. 覺得有用 (PT) 不等於就會用 (H1 不顯著)
3. 關鍵影響走「間接路徑」：PT → CF → IU (H2、H3 顯著)
4. 社會因素、個人/環境因素沒有直接推動 IU (H5、H6 不顯著)

Table 13. Path Coefficients, Bootstrap Estimates, Standard Deviation, T Statistics, and p-Values (Paper C)

Hypotheses	Coefficient	Bootstrap Mean	St. Dev.	T	p
H1: PT → IU	0.155	0.150	0.122	1.271	0.204
H2: PT → CF	0.766	0.766	0.047	16.405	0.000
H3: CF → IU	0.536	0.537	0.090	5.936	0.000
H4: PT → SF	0.405	0.408	0.076	5.346	0.000
H5: SF → IU	0.087	0.090	0.064	1.373	0.170
H6: PEF → IU	-0.004	-0.004	0.056	0.064	0.949
H7: PEF → PT	0.313	0.317	0.072	4.313	0.000

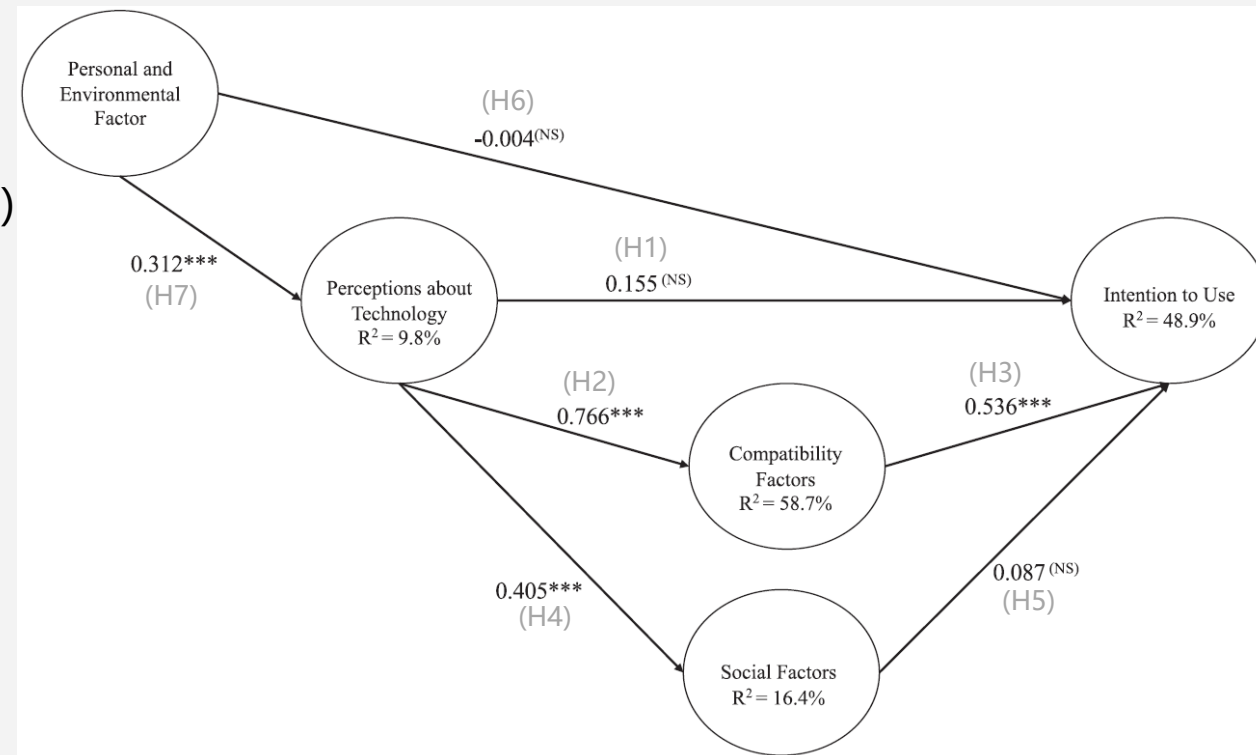


Fig. 2. HACAF' s structural model with R2 and path coefficients (***p < 0.001, (NS) p > 0.05). (Paper C)



落地清單：把 GenAI 變成「可控、可驗證」的工程流程

輸入管理 (Prompt / 資料)

- 不輸入敏感資料 (客戶資料、密碼、內部資訊)
- 提示用模板：目的 / 限制 / 輸出格式 / 測試條件

輸出驗證 (Code / 內容)

- 一律走 Code Review：AI 產出照樣開 PR
- 先跑測試再合併：單元測試 + 靜態檢查

品質門檻 (Quality Gates)

- 覆蓋率門檻 / 回歸測試必過
- 高風險模組 (資安、金流、權限) 必須人工複核

可追溯性 (Traceability)

- 重要需求/決策：保留 prompt、版本、review 記錄
- 出問題能回溯：哪次產出、哪次修改、誰核可



回扣論壇主題四面向

a. AI 輔助程式開發與程式碼品質

- 效率提升最明顯，但要靠 review / 測試門檻守住品質

b. AI 在軟體測試、品質保證與 DevOps 的角色

- 測試生成、自動化更強，但 gate 與監控會更重要

c. AI 介入軟體生命週期各階段

- 需求、設計、實作、測試、部署、維護：能力與普及度不均，但趨勢明確

d. 有 AI 的時代的工程師技能與倫理議題

- 訓練、信任、授權/智財、責任界線：會變成日常治理題